

Enforcing Orthogonal Updates with High-Rank Adapters for Continual Learning in LLMs

Ariel Haberman
ah5192@nyu.edu

Asif Mammadov
am14089@nyu.edu

Mario H. Garrido
mhg9251@nyu.edu

Tolga Dimlioglu
td2249@nyu.edu

Abstract

In this work, we focused on training a subset of layers of a Large Language Model (LLM) with forced orthogonality while performing continual learning of instructions. This is done to address possible limitations of O-LoRA (Wang et al., 2023), a previous method of continual learning which we hypothesize can suffer from performance bottlenecks. By training specific layers of the model with enforced orthogonality we hoped to prevent catastrophic forgetting and achieve better performance. However, we found that while O-LoRA does not actually suffer from these bottlenecks, by using rank we were able to satisfy a higher degree of orthogonality. On the other hand, we observed that this higher degree of achieved orthogonality does not directly benefit remembering the past task knowledge.

1 Introduction

Instruction tuning involves the additional training of Large Language Models (LLMs) using a dataset comprising pairs of instructions and their corresponding outputs. This process allows for the explicit fine-tuning of an LLM to follow instructions (Zhang et al., 2023). In a path to achieving a generalist model over time, the model should enhance its knowledge by adapting to new instructions. However, constantly retraining Large Language Models (LLMs) to incorporate every new instruction is impractical due to their immense size, making it a time-consuming and costly process. Continual Learning (CL) addresses this challenge by enabling models to continuously learn various tasks over time while retaining knowledge from previously learned ones (De Lange et al., 2022).

In the literature, there is strong evidence that LLMs suffer from catastrophic forgetting (CF) (Luo et al., 2024), a phenomenon where

neural networks tend to forget previously learned information when trained on new tasks. This makes the CL problem challenging. However, some remedies exist to mitigate the CF effect such as methods that replay the data seen on previous tasks, regularization of the model parameters, and expanding the network for accommodating more information (De Lange et al., 2022). Since we are working on LLMs, it is also inconvenient and computationally costly to continually fine-tune them end-to-end hence, the proposed approach has to be lightweight and efficient.

On this line, a CL method *O-LoRA* built on top of LoRA (Hu et al., 2022), a parameter-efficient fine-tuning (PEFT) technique that inserts adapters between the transformers layers, is introduced. In the O-LoRA method, the authors propose to apply orthogonal updates for each new task to the LoRA parameters to accommodate new task information without disrupting the parameter updates applied for the previously learned tasks (Wang et al., 2023). However, the orthogonality constraint is enforced on LoRA modules in different layers separately.

We argue that this layer-wise independent orthogonal updating scheme of O-LoRA does not guarantee that the task-specific update at each layer achieves orthogonality in a balanced manner. Ultimately, this might create a bottleneck issue due to some intermediate O-LoRA layers encapsulating less orthogonal task-specific directions and might cause performance limitations. To mitigate this, we propose applying orthogonality-enforced, task-specific updates to certain portion of the layers with higher rank, particularly focusing on the last ones since they contain more task-specific features. We believe that it is easier to accommodate tasks in orthogonal subspaces in this way.

2 Related Work

O-LoRA (Wang et al., 2023) which suggests employing orthogonal updates individually for each new task to adjust the LoRA parameters for parameter-efficient CL of instructions in LLMs. This method enables the integration of new task-related information without disturbing the parameter adjustments made for previously learned tasks. In terms of rehearsal-based CL, Continual-T0 (Scialom et al., 2022) stores %1 of the previously seen task samples and replays them during the training of the new task. In this way, it aims to retain the performance of the model on the previously learned tasks. We also looked at (de Masson D’Autume et al., 2019) establishes a standardized CL benchmark setup for consistent evaluation of different CL methodologies. It is based on the datasets used in (Zhang et al., 2015) and most of the works that came after this follow the same setup (Qin and Joty, 2021), (Razdaibiedina et al., 2022). The CL benchmark includes tasks such as topic classification and sentiment analysis.

3 Preliminaries

In this section, we formally define the CL setup. We also provide some background on the LoRA fine-tuning that we employ to fine-tune models and the O-LoRA method for CL.

3.1 Continual Instruction Learning Setup

In the continual learning setup, it is assumed that the learner (with parameters Θ) experiences a stream of tasks $T_1, T_2, \dots$ that arrive sequentially (Farajtabar et al., 2020). When the learner is being trained on T_i it can only access the samples of the current task, and cannot revisit the previous task samples or foresee the upcoming task samples. In continual instruction learning, task samples arrive as triples: $T_i = \{(x_n^i, y_n^i, I_i)\}_{n=1}^{N_i}$ where x_n^i, y_n^i, I_i are input tokens, target tokens and instruction prompt of sample n of task i respectively. The learner’s goal while being trained on task t is to optimize the following objective:

$$\max_{\Theta} \sum_{i=1}^t \sum_{n=1}^{N_i} \log p_{\Theta}(y_n^i | I_i, x_n^i) \quad (1)$$

However, since the learner has only access to the current task samples, directly optimizing the objective above is challenging. Hence, continual learning algorithms should aim to retain previously

learned task information while being able to accommodate new tasks (Tiwari et al., 2022).

Although some previous works based their Continual Instruction Learning setup (Scialom et al., 2022; Yin et al., 2022; Wang et al., 2023) on the T-5 model (Raffel et al., 2020), we believe directing our attention towards decoder-only style models might be more relevant to the current landscape of the NLP research, as these models are more popular with chat-bot applications. We use Facebook’s OPT model family (Zhang et al., 2022), particularly the 2.7B model.

3.2 LoRA Fine-tuning

Let us first give a background for LoRA (Low-Rank Adaptation) (Hu et al., 2022) method which is a parameter-efficient fine-tuning method that can be used to fine-tune LLMs on downstream tasks. Aghajanyan et al. (2021) demonstrated that when LLMs are fine-tuned on a certain task, the weight changes have a low intrinsic dimension. Motivated by this work, (Hu et al., 2022) represents the weight update ΔW as a low-rank decomposition of two matrices A and B : $\Delta W = AB$. Here, $A \in R^{d \times r}$, $B \in R^{r \times k}$ and $\Delta W \in R^{d \times k}$ where $r \ll d, k$. In this way, the burden of fine-tuning a large model on a downstream task in an end-to-end manner is alleviated. The empirical results in (Hu et al., 2022) show that with LoRA fine-tuning, it is possible to match the end-to-end training performance with significantly less number of trainable parameters. In all of the baseline and CL algorithms, we adopt LoRA as our fine-tuning method in order to have a lightweight fine-tuning method and to ensure a fair comparison of other baselines with O-LoRA.

3.3 O-LoRA

In O-LoRA, it is proposed to find task-specific orthogonal subspaces every time a new task is learned so that the new task information can be accommodated by the model without interfering with the previously learned task subspace. For each new task t , a new set of LoRA parameters are introduced A_t, B_t . The subspace of the task is approximated as the column space of A_t and, B_t is treated as the linear weighting coefficients for the column vectors of A_t . To maintain the orthogonality of two subspaces, it is required to satisfy orthogonality between the basis vectors of the two subspaces. So

ideally, when orthogonality between tasks i and j are achieved, we get $O_{i,t} = A_i^T A_t = 0$. To encourage this orthogonality condition when training the model on task t , the authors introduce the following regularization term to the language modeling objective:

$$\lambda_1 \sum_{i=1}^{t-1} \sum_{j,k} |(A_i^T A_t)_{j,k}| \quad (2)$$

Here t is the current task index; j, k are the row and column indices for the matrix product $A_i^T A_t$ and, λ_1 is the strength of the regularization. In the paper, an empirically obtained optimal value of λ_1 is mentioned as 0.5.

4 Our Method

Because O-LoRA enforces orthogonality on different layers independently, the captured orthogonality might be imbalanced across layers, creating a bottleneck issue and ultimately degrading the performance. We argue that by focusing on certain layers and allowing higher LoRA rank (r) on them i) it can be easier to accommodate tasks in orthogonal spaces ii) we can ensure more balanced orthogonality across layers as we have less number of trainable LoRA layers with higher column rank giving flexibility in seeking orthogonality. Following our claims, we propose to employ O-LoRA on a certain proportion of the final model layers. We particularly test two settings: O-LoRA finetuning of the final 50% of model layers with doubling the rank ($r = 16$) and final 25% of the model layers with quadrupling the rank ($r = 32$). The rank values are set so that the total number of trainable parameters stays the same as the vanilla O-LoRA with $r = 8$.

5 Methodology

Here we describe the details of our experiments.

5.1 Experiment 1

In the experiment set 1, we report the ROUGE-1 (Recall-Oriented Understudy for Gisting Evaluation-1)(Lin, 2004). We present the results in a table format by sharing the task-level and final average ROUGE-1 scores on the test set obtained by the model at the end of continual training.

5.2 Experiment 2

In experiment 2, we dive deep into the task-specific LoRA parameters and investigate how much or-

thogonality is satisfied on average. We quantify this by first calculating the average cosine similarity score across all task-specific subspace vectors and then taking $\arccos$ of it to have a more interpretable score in degrees($^\circ$). More formally, for each layer l we calculate the following score:

$$\arccos \left(\frac{2}{K(K-1)} \sum_{i=2}^K \sum_{j=1}^{i-1} \text{avg} (|\bar{A}_{l,i}^T \bar{A}_{l,j}|) \right) \quad (3)$$

where K is the total number of tasks in the sequence, $\bar{A}_{l,i}$ is the normalized LoRA A parameters at layer l after the model is trained on task i and we symbolically write avg to express taking the average of the absolute values of the inner matrix product entries. This experiment helps us investigate how much orthogonality is achieved and how the achieved orthogonality varies across different model layers. We would like to remark that this metric is developed by us.

6 Experiments

In this section, we describe the details of the training, datasets and tasks we use, baseline methods and our method.

6.1 Training Details

In all the methods, we employ LoRA (Hu et al., 2022) fine-tuning with $r = 8$, $\alpha = 8$ and $\text{dropout} = 0.1$ on Query and Value heads unless otherwise stated. In the CL setup, we train the model on each new task for 2 epochs using AdamW optimizer (Loshchilov and Hutter, 2018) with a constant learning rate of $5e - 5$ and weight decay of $1e - 2$. We set the batch size as 4 on each GPU and we use the DeepSpeed (Rasley et al., 2020) stage 2 for distributed training with optimizer state offloading.

6.2 Dataset and Task Orders

We assess our method using datasets from (Zhang et al., 2015) which is commonly adopted by the community as the CL benchmark for language model. We use two different task orders with a task length of 4. We adopt the first order from (Wang et al., 2023) and we create the second order ourselves.

- Order 1: dbpedia $\rightarrow$ amazon $\rightarrow$ yahoo $\rightarrow$ agnews

- Order 2: mnli $\rightarrow$ boolqa $\rightarrow$ agnews $\rightarrow$ yelp

Order 1 contains three topic classification tasks (dbpedia, yahoo, agnews) and one sentiment analysis task (amazon). Order 2 has more variety and it includes NLI (MNLI), answering a yes-no question (BoolQA), topic classification (agnews) and sentiment analysis (yelp). The details regarding the datasets can be found in Appendix A.1.

6.3 Baselines

Here we describe the baseline methods in addition to O-LoRA that we compare with our method.

Joint Training: The joint training of a model on the different tasks does not fall into the category of continual instruction tuning. However, it determines the upper bound values for the ROUGE-1 scores. The results of this baseline are collected by simply concatenating all the instruction-tuning tasks in the task sequence, shuffling the concatenated task samples, and then fine-tuning the model on this joint dataset. When presenting the results in the table, we refer to this baseline as *Joint*.

Sequential Training: Sequential training is achieved by continually instruction-tuning the model on a sequence of different tasks. This training setting gives us lower bound scores since we observe how much of the previously acquired information on the tasks is forgotten without using any technique to alleviate catastrophic forgetting when the model is sequentially trained on different tasks. We refer to this setting as *Seq. (Basic)*. We have another baseline for sequential training scheme. In this version, after the model is trained on a task, the LoRA parameters are merged to model layers ($W \leftarrow W + AB$) and a new set of LoRA parameters are initialized for the upcoming task. We refer to this baseline as *Seq. (Merged)*.

Rehearsal: Although it violates the crucial constraint of not accessing the past task samples in continual learning, the rehearsal-based methods store a certain amount of samples of the previously seen tasks. We follow the recipe described in (Scialom et al., 2022) and store 1% of the samples from the previous tasks. The stored samples are replayed with the samples of the current task. This approach directly assists the learner in remembering past tasks.

7 Experimental Results

First, we present the results of experiment 1 on two different task orders described earlier. The ROUGE-1 scores are calculated on the test set after the model is continually trained on the whole task sequence.

Method	dbpedia	amazon	yahoo	agnews	Avg.
Joint	98.52	73.07	73.97	90.15	83.93
Rehearsal	98.01	70.63	72.04	90.89	82.89
O-LoRA	77.05	44.16	51.80	90.32	65.83
O-LoRA (last 50%)	72.83	65.07	51.40	88.51	69.45
O-LoRA (last 25%)	59.60	0.05	27.15	88.40	43.80
Seq. (Merged)	69.98	46.43	48.46	90.87	63.94
Seq. (Basic)	36.04	56.56	44.46	91.41	57.11

Table 1: Test set ROUGE-1 scores on Order 1

We observe that in Order 1, the Rehearsal approach is the only CL method that almost matches the joint training performance. Our proposed O-LoRA (last 50%) setting performs better than the vanilla O-LoRA whereas the last 25% setting significantly underperforms not only the vanilla O-LoRA but all the other baselines.

Method	mnli	boolqa	agnews	yelp	Avg.
Joint	74.84	65.98	90.62	77.22	79.02
Rehearsal	52.80	63.15	89.68	77.43	72.03
O-LoRA	54.46	62.89	87.21	77.08	71.66
O-LoRA (last 50%)	45.74	63.09	87.23	75.94	68.82
O-LoRA (last 25%)	30.97	49.97	52.28	73.39	51.93
Seq. (Merged)	47.08	63.18	88.63	78.22	70.29
Seq. (Basic)	35.32	64.29	86.58	77.55	66.21

Table 2: Test set ROUGE-1 scores on Order 2

In Order 2, the Rehearsal and vanilla O-LoRA perform similarly but our proposed settings fall behind the two. Again, the last 25% setting underperforms significantly. Also, surprisingly, the Sequential LoRA training with merged parameters performs very similarly to O-LoRA and our methods (which is the case both in Order 1 and Order 2). Seeing these interesting results, we move on to presenting experiment 2 findings to gain insights about the captured orthogonality.

As mentioned earlier, we quantify the orthogonality as arccos of the average cosine similarity between the task subspaces as defined in expression 3.

From Figures 1 and 2; we first observe that in the vanilla O-LoRA method, there is almost

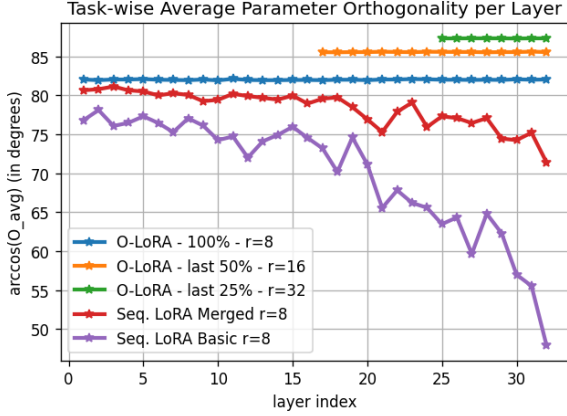


Figure 1: Per-Layer captured orthogonality on average for Order 1

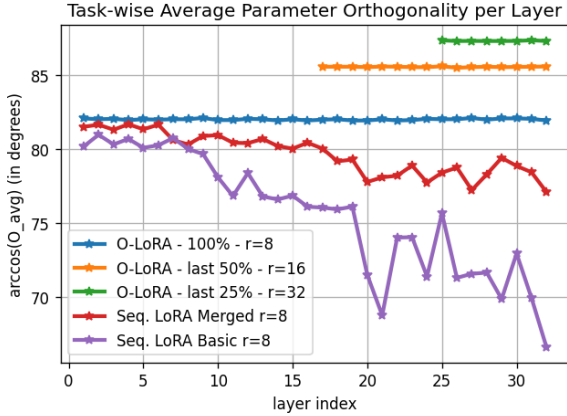


Figure 2: Per-Layer captured orthogonality on average for Order 2

no variation in the captured orthogonality across different layers which falsifies our initial claim. But we also see that when increasing the rank and focusing on certain parts of the model with orthogonality updates, it is possible to achieve better orthogonality across task subspaces. This verifies our second claim. However, having better orthogonality does not directly benefit the model’s knowledge retention capabilities in CL. In fact, sometimes hinders as can be seen from the ROUGE-1 scores in Tables 1 and 2.

Another interesting observation here is that, the Seq. (Merged) method already has a decent level of orthogonality without any explicit orthogonality-enforcing updates and it retains almost the same level of orthogonality across different layers. For Seq. (Basic) however, the orthogonality is weaker as can be anticipated.

8 Conclusion

In this work, we test two hypotheses about the O-LoRA method (Wang et al., 2023) to improve the learner’s past task information retention capabilities in the CL setup. We observe that our initial hypothesis about the O-LoRA method not ensuring orthogonality across different layers in a balanced manner and causing bottleneck problems is falsified as demonstrated in experiment set 2. On the other hand, our intuition about satisfying orthogonality in a subset of final layers with higher rank is correct but the achieved orthogonality does not directly translate to improvement in the model performance and past task knowledge retention. Our results raise the following question: to what extent orthogonality across different task subspaces is important to mitigate forgetting the information acquired in the past?

9 Future Work

Currently, our analysis is carried out on a relatively short task sequence, with a length of four. It would be interesting to carry out a similar analysis on longer task sequences. On top of longer task sequences, the experiments could be replicated with different model sizes to have more concrete and informative conclusions.

10 Ethical Considerations

At the time we have not been able to think of any moral or ethical problems with this project. This of course could change if we changed the tasks or datasets used.

11 Limitations

The experiments were conducted using relatively small large language models (LLMs), specifically Facebook OPT 2.7B. With access to greater resources, we could have experimented with models having a higher number of parameters. By following the task order from (Wang et al., 2023) one of our orders consisted of three of the same task and one different whereas our other order consisted of four different tasks. This meant we were not able to see if task similarity made any difference the way we could if the first order were all the same task.

12 Contribution

All authors of the paper contributed to the development process for this paper. Ariel Haberman prepared dataloader scripts, Asif Mammadov implemented the rehearsal method, Mario Garrido implemented the O-LoRA Method, and Tolga Dimlioglu prepared the overall codebase and integrated the tools. The experiments were run and results were collected by all the authors.

References

- Armen Aghajanyan, Sonal Gupta, and Luke Zettlemoyer. 2021. [Intrinsic dimensionality explains the effectiveness of language model fine-tuning](#). In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 7319–7328, Online. Association for Computational Linguistics.
- Matthias De Lange, Rahaf Aljundi, Marc Masana, Sarah Parisot, Xu Jia, Aleš Leonardis, Gregory Slabaugh, and Tinne Tuytelaars. 2022. [A continual learning survey: Defying forgetting in classification tasks](#). *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(7):3366–3385.
- Cyprien de Masson D’Autume, Sebastian Ruder, Lingpeng Kong, and Dani Yogatama. 2019. Episodic memory in lifelong language learning. *Advances in Neural Information Processing Systems*, 32.
- Mehrdad Farajtabar, Navid Azizan, Alex Mott, and Ang Li. 2020. Orthogonal gradient descent for continual learning. In *International Conference on Artificial Intelligence and Statistics*, pages 3762–3773. PMLR.
- Edward J Hu, yelong shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. [LoRA: Low-rank adaptation of large language models](#). In *International Conference on Learning Representations*.
- Chin-Yew Lin. 2004. Rouge: A package for automatic evaluation of summaries. In *Proceedings of the Association for Computational Linguistics*, pages 74–81.
- Ilya Loshchilov and Frank Hutter. 2018. [Fixing weight decay regularization in adam](#).
- Yun Luo, Zhen Yang, Fandong Meng, Yafu Li, Jie Zhou, and Yue Zhang. 2024. [An Empirical Study of Catastrophic Forgetting in Large Language Models During Continual Fine-tuning](#). *arXiv preprint*. ArXiv:2308.08747 [cs].
- Chengwei Qin and Shafiq Joty. 2021. Lfpt5: A unified framework for lifelong few-shot language learning based on prompt tuning of t5. In *International Conference on Learning Representations*.
- Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of machine learning research*, 21(140):1–67.
- Jeff Rasley, Samyam Rajbhandari, Olatunji Ruwase, and Yuxiong He. 2020. Deepspeed: System optimizations enable training deep learning models with over 100 billion parameters. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 3505–3506.
- Anastasia Razdaibiedina, Yuning Mao, Rui Hou, Madihan Khabsa, Mike Lewis, and Amjad Almahairi. 2022. Progressive prompts: Continual learning for language models. In *The Eleventh International Conference on Learning Representations*.
- Thomas Scialom, Tuhin Chakrabarty, and Smaranda Muresan. 2022. [Fine-tuned language models are continual learners](#). In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 6107–6122, Abu Dhabi, United Arab Emirates. Association for Computational Linguistics.
- Rishabh Tiwari, Krishnateja Killamsetty, Rishabh Iyer, and Pradeep Shenoy. 2022. Gcr: Gradient coreset based replay buffer selection for continual learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 99–108.
- Xiao Wang, Tianze Chen, Qiming Ge, Han Xia, Rong Bao, Rui Zheng, Qi Zhang, Tao Gui, and Xuanjing Huang. 2023. [Orthogonal subspace learning for language model continual learning](#). In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 10658–10671, Singapore. Association for Computational Linguistics.
- Wenpeng Yin, Jia Li, and Caiming Xiong. 2022. [ConTinTin: Continual learning from task instructions](#). In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3062–3072, Dublin, Ireland. Association for Computational Linguistics.
- Shengyu Zhang, Linfeng Dong, Xiaoya Li, Sen Zhang, Xiaofei Sun, Shuhe Wang, Jiwei Li, Runyi Hu, Tianwei Zhang, Fei Wu, and Guoyin Wang. 2023. [Instruction tuning for large language models: A survey](#). *Preprint*, arXiv:2308.10792.
- Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona Diab, Xian Li, Xi Victoria Lin, et al. 2022. Opt: Open pre-trained transformer language models. *arXiv preprint arXiv:2205.01068*.
- Xiang Zhang, Junbo Zhao, and Yann LeCun. 2015. [Character-level convolutional networks for text classification](#). In *Advances in Neural Information Processing Systems*, volume 28. Curran Associates, Inc.

A Appendices

A.1 Details on Datasets

- DBPedia ontology dataset: the task is to determine the topic of the input text from 14 possible labels. "Company", "Educational Institution", "Artist", "Athlete", "Office Holder", "Mean of Transportation", "Building", "Natural Place", "Village", "Animal", "Plant", "Album", "Film" or "Written Work". The dataset contains 7,600 development and test examples each and 14,000 train.
- Amazon Reviews: the task for this dataset is sentiment analysis with a choice of 5 labels. "very negative", "negative", "neutral", "positive" or "very positive". The dataset contains 7,600 development and test examples each and 5,000 train.
- Yahoo! Answers: the task here is to determine the topic of the input text from 10 possible labels. "Society & Culture", "Science & Mathematics", "Health", "Education & Reference", "Computers & Internet", "Sports", "Business & Finance", "Entertainment & Music", "Family & Relationships", or "Politics & Government". The dataset contains 7,600 development and test examples each and 10,000 train.
- Ag News: the task for this dataset is to determine the topic of the input text from 4 possible labels. "World", "Sports", "Business", or "Science or Technology". The dataset contains 7,600 development and test examples each and 10,000 train.
- MNLI: the task is to predict if in a sentence pair the premise entails or contradicts the hypothesis or does neither. There are 3 possible labels "entailment", "contradiction" and "neutral". The dataset contains 7,600 development and test examples each and 3,000 train.
- BoolQA: the task is to determine the answer to a yes or no question based on a given passage. There are two labels, "true" and "false". The dataset contains 3,270 development and test examples each and 2,000 train.
- Yelp Reviews: the task here is sentiment analysis by way of predicting the number of stars a user has left with their review. There are five

possible labels "very negative", "negative", "neutral", "positive", and "very positive". The dataset contains 7,600 development and test examples each and 5,000 train.

The prompts are given in one of two ways. Either just an instruction that describes a task or paired with an input. The prompt itself is "Write a response that appropriately completes the request." The instruction contains the task itself and the list of possible outputs. Each dataset is in the form of a JSON containing the instruction, output and input where the instruction is the task, output the label and input the sentence.

A.2 Training Infrastructure

For running our experiments, we used NYU HPC, particularly the RTX8000 nodes. We also used 2 x RTX 3090 GPUs.